java

ingegneria del software

LEZIONE 6 MARZO

`system.out.println();`
scrive una stringa ma cosi è lungo come comando, lo possiamo abbreviare con:
`IO.println();` → (sul mio vscode non funziona)

In java tutte le variabili dichiarate sono di default inizializzate a 0

Solitamente si crea un file per ogni classe

Si può compilare con il tasto run oppure con “javac HelloWorld.java” (compilazione), in questo caso il compilatore genera il corrispondente file eseguibile chiamato “HelloWorld.class”, e poi se non ci sono errori posso scrivere “java HelloWorld” (esecuzione).

```
import java.time.LocalDate;
```

```
public class HelloWorld {
private static int valore = 0;
private int contatore;
private String saluto="Ciao";
private static LocalDate data= LocalDate.now();
```

```
public static void main(String[] args){
System.out.println("Hello World.");
```

```
HelloWorld h=new HelloWorld();
h.stampe();
}
```

```
private void stampe(){
valore++;
System.out.println(data);
IO.println("ciao ciao");
IO.println("valore : " +valore);
System.out.println(saluto + ", Ingegneria del softw
are.");
contatore++;
System.out.println("contatore: " +contatore);
```

```
/** HelloWorld hello;
*hello= new HelloWorld(); lo tolgo perchè l'ho crea
to nel main
*hello.contatore++;
*System.out.println("contatore: " +hello.contator
e);
*HelloWorld hello1;
*hello1= new HelloWorld();
*System.out.println(hello1.contatore);
*/
}
}
```

```
System.out.println("Hello World."); serve per stampare qual
cosa
```

se scrivo sout mi spunta sotto il suggerimento per System.out.println().

Questa è una chiamata al metodo println che viene formata attraverso l’istanza di out

Un altro modo per stampare è:

```
IO.println("ciao ciao");
```

La parola static nel main indica che il metodo vive all’interno della classe, quando metodi o attributi sono static operiamo direttamente nella classe, non c’è bisogno di istanziare la classe ma operiamo con essa.

per dichiarare un valore:

```
private static int valore=0;
```

- private è la visibilità della variabile per le altre classi
- static indica che il la variabile vive dentro la classe

fin quando non uso la variabile creata spunta sottolineata.

```
IO.println("valore : "+valore);
```

Con alcuni editor può essere mostrata parte di testo in grigio che non abbiamo scritto, come in System.out.printl(x:) in cui la x non è effettivamente presente nel codice ma è un suggerimento, viene scritto automaticamente, se apro il file con un editor differente non lo vedrò scritto.

Posso anche dichiarare una variabile senza usare static.
ES: private int contatore;

ma se provo ad utilizzarlo dentro al main spunterà sottolineato in rosso come un errore perchè non possiamo usare una variabile non static all'interno dell'ambiente main che è static. Se lo voglio utilizzare devo creare l'istanza e inizializzarla in questo modo:

```
HelloWorld hello;
hello = new HelloWorld();
```

per richiamare una variabile appartenente alla classe dopo aver creato l'istanza scrivo:

```
hello.contatore++;
```

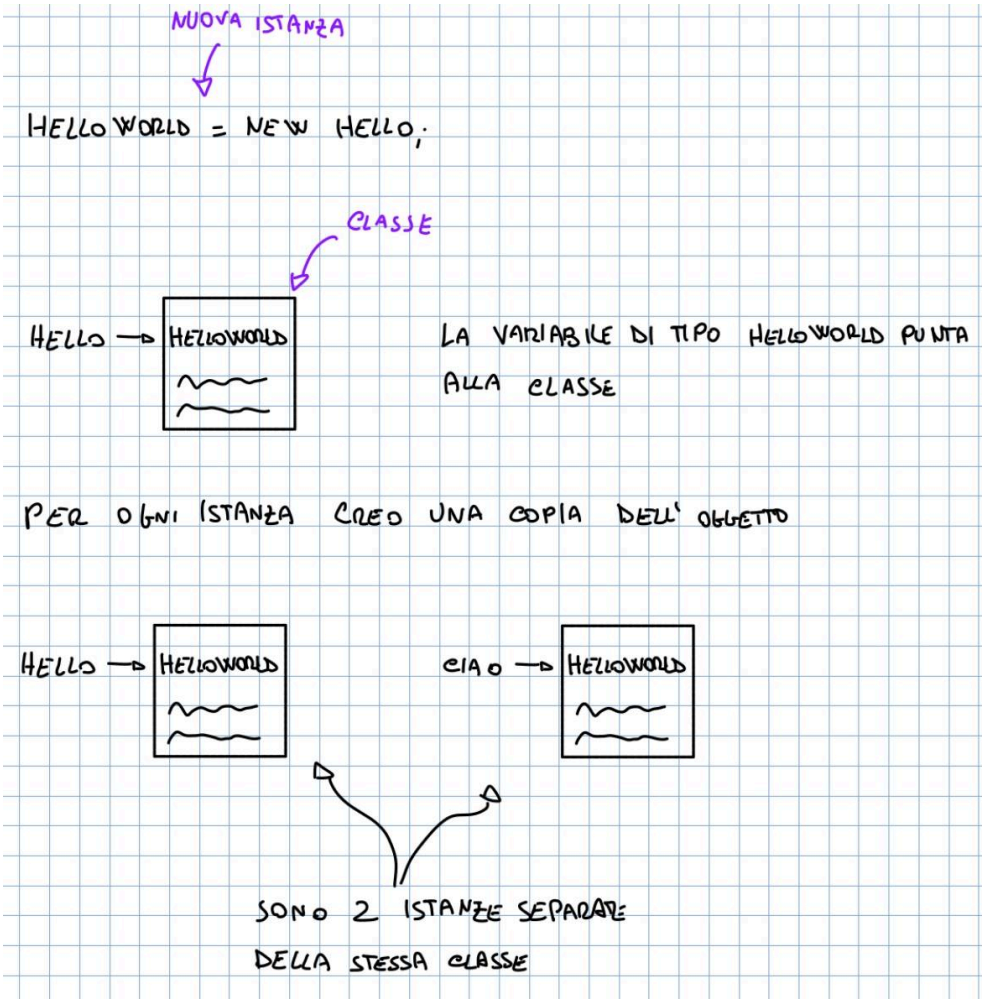
devo mettere sempre "hello." davanti alla variabile (in questo caso contatore) per utilizzarla:

```
System.out.println("contatore: " + hello.contatore);
```

se parto da un'istanza e da quella prendo un attributo che è dentro la classe, quindi static, accedo all'unico contenuto che è dentro la classe non viene duplicato. Quindi non spunta un errore se scriviamo ES. hello.valore) ma non è corretto.

CREAZIONE ISTANZA

La variabile chiamata “hello” punta alla parte di memoria in cui è stata creata, nell’istanza creata troveremo la variabile. In un’altra parte di memoria riservata alla classe
HelloWorld troviamo valore.




```
HelloWorld hello1;
hello1=hello;
System.out.println(hello1.contatore);
```

in hello 1 ricopiamo “il puntatore” che avevamo in hello, puntano alla stessa parte di memoria (stessa classe)

se invece scrivo:

```
HelloWorld hello1;
hello1= new HelloWorld();
System.out.println(hello1.contatore);
```

si crea un nuovo spazio di memoria (nuova istanza della stessa classe, come se fosse una copia)

```
Le variabili di tipo numerico non inizializzate valgono 0.
```

Se creo un'altro metodo deve essere poi richiamato nel main, ma si deve sempre creare un'istanza per il metodo.

```
HelloWorld h=new HelloWorld();
h.stampe();
```

se abbiamo più parametri possiamo concatenarli tra di loro in println() tramite il

```
private String saluto="Ciao";
System.out.println(saluto + ", Ingegneria del software.");
```

Se usiamo dei tipi primitivi vengono (int,float,double,char...) iniziano con la lettera minuscola mentre ad esempio altri come String che in realtà è una classe inizia con la maiuscola, questo serve per distinguere classi (che forniscono dei tipi) da tipi primitivi.

```
private static LocalDate data = LocalDate.now();
```

serve per stampare la data, in alto troveremo: import java.time.LocalDate; serve per comunicare al compilatore dove si trova la classe LocalDate.

import indica dove andare a cercare nelle cartelle, → dentro java → dentro time → prendi LocalDate

Perchè si deve scrivere import per LocalDate ma non per IO, System e String se sono anche queste delle classi?

Per convenzione, tutti i tipi definiti nelle librerie standard di java (java.lang) non devono essere specificate con import, LocalDate non fa parte di quella libreria quindi si deve specificare.

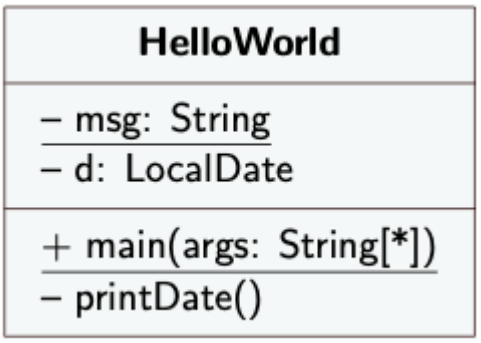
se scrivo import java.time. *; significa che posso usare tutto quello che c'è nella cartella se non uso* importo il tipo specifico.

All'esame se non si ricorda import non è un problema!!!

"@param args" sono i parametri passati al main all'avvio della classe

I commenti conviene scriverli fuori dai metodi come “descrizione”, quindi conviene scriverli prima di classi e metodi.

UML: descrive il sistema software di ciò che si sta sviluppando, viene utilizzato per descrivere delle classi, relazioni tra di esse. Viene raffigurato come un rettangolo.



I simboli sono:

- " - " " + " e " *" (asterisco) per indicare la visibilità delle variabili, - indica private, + indica public e* indica protected
- La sottolineatura indica se una variabile è static

Abbiamo quindi il nome della classe in alto, poi le variabili e il tipo e infine i ,metodi con i loro eventuali parametri e i tipi di ritorno

Ad esempio qui abbiamo: La classe HelloWorld e i suoi attributi sono: msg che è private è di tipo string ed è static, abbiamo anche d che è private e di tipo LocalDate ma non è static
i metodi invece sono il main che è static e public e printDate che è private

Quando ho un diagramma UML ciò che non è presente nel diagramma, non significa che non esiste ma semplicemente che non è stato espresso all’interno di esso.

final definisce un campo o una variabile che non può essere assegnata più di una volta (una costante). Una classe final non può essere ereditata, un metodo final non può essere ridefinito (override).

new permette di creare un’istanza di una classe.
static è usata per dichiarare un campo o un metodo appartenente alla classe (e non all’istanza).

CARATTERISTICHE DEL SOFTWARE

Ha delle caratteristiche particolari:

```
È modificabile perchè non ha parti fisiche (non è costituito da atomi).
Posso
prendere il codice sorgente e modificare nomi di variabili ecc... ma anche
modificare il codice eseguibile.
```

Se un software è di successo può esserci la necessità di modificarlo:

```
Per adattarlo ad una realtà che cambia (mutate esigenze).
Le richieste di estensione aumentano al crescere del successo.
```

Il sistema software sopravvive all'hardware, perché l'hardware può essere sostituito ma il software può comunque essere eseguito.

Il software quindi è modificabile e manutenibile. La manutenibilità è diversa dalla modificabilità.

QUALITÀ DEL SOFTWARE

Per valutare la qualità:

```
Correttezza: il sistema software aderisce allo scopo ed è conforme alle
specifiche (requisiti). Documento dei requisiti scritto correttamente.
Efficienza: nessuno spreco di memoria e risorse RAM e processore).
Manutenibilità: facilità a introdurre modifiche (poco spreco di risorse e
tempo, si aumenta l'efficienza).
```

Modificare il software è proprio uno degli obiettivi principali quando si parla di grandi software, se si produce una versione non modificabile perchè c'è una bassa manutenibilità è un problema, si deve rifare.

Come lo rendo manutenibile se non so cosa modificherò in seguito? La risposta è design Pattern (soluzioni collaudate per riusabilità).

```
Dependability Affidabilità e sicurezza): sicurezza (security, difesa dei
dati; e safety, difesa del sistema e delle persone) e affidabilità
(reliability,
probabilità di operare senza guasti). Può eseguire per tanto tempo senza
errori, più è il tempo più è affidabile. Ci sono delle misure che
stabiliscono
l'affidabilità. La sicurezza è data da due fattori : Safety e Security.
Hanno
un significato diverso:
SECURITY: capacità di proteggere i dati e non perderli.
SAFETY: in termini di sicurezza/salute delle persone. Non mette in pericolo
la vita delle persone.
Usabilità: capacità di fornire all'utente condizioni di uso efficaci.
Permette
all'utente di operare in modo facile rispetto a quello che deve fare.
```

Sistema software corretto

Domanda esame: come verifico se un sistema software è corretto?
Un sistema software da risultati corretti se soddisfa le specifiche o requisiti richiesti. Per sapere se li soddisfa si fanno dei test. Si esegue e si valutano i risultati, se uguali a quelli dei requisiti allora il sistema si comporta in modo corretto

Per essere corretto si deve verificare non solo se rispetta i requisiti ma anche se è corretto per il cliente.